

18.3

Algorithme de Boyer-Moore

NSI TERMINALE - JB DUTHOIT

18.3.1 Généralités

- Dû à Robert S. Boyer et J. Strother Moore – 1977
- utilisé le plus souvent dans les éditeurs de texte (tel quel ou optimisé)

18.3.2 Principe

L'algorithme de Boyer-Moore se base sur les caractéristiques suivantes :

- L'algorithme effectue un **pré-traitement du motif**. Cela signifie que l'algorithme "connait" les caractères qui se trouvent dans le motif
- On commence la **comparaison motif-chaine par la droite** du motif. Par exemple pour le motif CGGCAG, on compare d'abord le G, puis le A, puis C...on parcourt le motif de la droite vers la gauche
- Dans la méthode naïve, les décalages du motif vers la droite se faisaient toujours d'un "cran" à la fois. L'intérêt de l'algorithme de Boyer-Moore, c'est qu'il permet, dans certaines situations, d'effectuer un décalage de plusieurs crans en une seule fois. Pour cela, il existe deux règles :

– La **règle du mauvais caractère** :

Lors de la comparaison de droite à gauche, si un caractère du texte ne correspond pas au motif, ce caractère du texte est appelé le "**mauvais caractère**". Pour déterminer de combien de cases décaler le motif vers la droite, on applique l'un de ces deux cas :

- * Cas 1 : Le mauvais caractère n'existe pas dans le motif. On décale le motif entier pour le placer juste après ce caractère (puisque aucune superposition n'est possible avec lui).
- * Cas 2 : Le mauvais caractère existe dans le motif. On fait glisser le motif vers la droite de façon à aligner la dernière occurrence de ce caractère dans le motif avec le mauvais caractère du texte.

L'objectif de cette règle est d'effectuer les sauts les plus grands possibles en toute sécurité, sans risquer de rater une correspondance valide.

– La **règle du bon suffixe**, que nous n'allons pas étudier dans ce chapitre. Je vous donne cependant le principe :

La règle du bon suffixe est la deuxième règle de l'algorithme de Boyer-Moore. Elle intervient quand vous avez réussi à faire correspondre plusieurs lettres à la fin du motif avant de rencontrer une erreur.

Plutôt que d'utiliser la lettre qui a échoué (mauvais caractère), on utilise la partie du motif qui a réussi (le suffixe). On cherche si ce bloc de lettres existe ailleurs dans le motif pour effectuer un saut. 2 cas :

- * Si le suffixe qui a fonctionné se retrouve ailleurs à l'intérieur du motif, on décale le motif pour aligner cette occurrence précédente avec le suffixe du texte.

- * Si le suffixe complet n'existe pas ailleurs, on cherche si une partie de la fin du suffixe correspond au début (préfixe) du motif. Si oui, on aligne le début du motif sur cette fin de suffixe.

L'algorithme de Boyer-Moore complet calcule les deux sauts (celui du mauvais caractère et celui du bon suffixe) et choisit systématiquement le plus grand des deux.

Etape 1 :

CAAGCGCACAAGACGCGGCAGACCTTCGTTATAGGCGATGATTT
ACG

A et G ne correspondent pas. On sait qu'il y a un A dans le motif... On décale donc de deux crans vers la droite.

Etape 2 :

CAAGCGCACAAGACGCGGCAGACCTTCGTTATAGGCGATGATTT
ACG

C et G ne correspondent pas. On sait qu'il y a un C dans le motif... On décale donc d'un cran vers la droite.

Etape 3 :

CAAGCGCACAAGACGCGGCAGACCTTCGTTATAGGCGATGATTT
ACG

G et G se correspondent, ainsi que C et C. En revanche, G et A ne se correspondent pas...G n'est pas dans le "reste" de la clé...on décale donc de 1 cran vers la droite.

Etape 4 :

CAAGCGCACAAGACGCGGCAGACCTTCGTTATAGGCGATGATTT
ACG

G et C ne correspondent pas. C est présent dans la clé, donc on décale d'un cran..

... et ainsi de suite jusqu'au moment où l'on trouve (ou pas !) une correspondance parfaite.

18.3.3 Exemple

Dans cet exemple précis, les données d'entrée sont les suivantes :

Le texte p x y o t a v a t a v a t

Le motif v a t a v a

Le tableau trace étape par étape (colonne "temps") les index comparés `i_texte` et `i_motif` et le résultat de chaque comparaison pour trouver la correspondance finale.

texte :	p	x	y	o	t	a	v	a	t	a	v	a	t	i_texte	i_motif	comparaison	temps
motif :	v	a	t	a	v	a								5	5	'a' == 'a'	1
	v	a	<u>t</u>	a	v	a								4	4	't' != 'v'	2
			v	a	t	a	v	a						7	5	'a' == 'a'	3
			v	a	t	a	v	a						6	4	'v' == 'v'	4
			v	a	t	a	v	a						5	3	'a' == 'a'	5
			v	a	t	a	v	a						4	2	't' == 't'	6
			v	a	t	a	v	a						3	1	'o' != 'a'	7
				v	a	<u>t</u>	a	v	a					8	5	't' != 'a'	8
					v	a	t	a	v	a				11	5	'a' == 'a'	9
					v	a	t	a	v	a				10	4	'v' == 'v'	10
					v	a	t	a	v	a				9	3	'a' == 'a'	11
					v	a	t	a	v	a				8	2	't' == 't'	12
					v	a	t	a	v	a				7	1	'a' == 'a'	13
					v	a	t	a	v	a				6	0	'v' == 'v'	14

Au total, l'algorithme a donc réalisé 14 comparaisons de caractères pour trouver la solution, au lieu de 18 avec un algorithme simple (décalage d'une case à la fois en lisant le motif à l'envers)

☛ Sur de longs textes de plusieurs milliers de caractères, cette capacité à "sauter" des blocs entiers d'un seul coup (plutôt que de glisser péniblement d'une case à la fois) permet de réduire considérablement le temps d'exécution global.

● Exercice 18.11

On considère le texte suivant :

ATAACAGGAGTAAATAACGGTGGGAGTA

et le motif CGGTG

1. Essayer de tête d'imaginer l'algorithme naïf et déterminer en combien de comparaisons il trouvera l'indice de l'occurrence.
2. Détailler l'algorithme de Boyer Moore sur cet exemple. Combien de comparaisons sont nécessaires ?

```
texte : A T A A C A G G A G T A A A T A A C G G T G G G A G T A i_t i_m comp. tps
```

motif : C G G T G

1

2

3

4

5

6

7

8

9

10

Exercice 18.12

On considère le texte suivant :
ABRACADABRA
et le motif DAB

1. Essayer de tête d'imaginer l'algorithme naïf et déterminer en combien de comparaisons il trouvera l'indice de l'occurrence.
2. Détailler l'algorithme de Boyer Moore sur cet exemple. Combien de comparaisons sont nécessaires ?

texte : A B R A C A D A B R A tps	
motif : D A B	1
	2
	3
	4
	5
	6

18.3.4 Pré-traitement

Une solution de pré traitement

Le pré-traitement du motif consiste à garder dans une liste en mémoire la position de la dernière occurrence de chaque caractère distinct du motif (sauf le dernier , car il est utilisé pour comparer)

Exercice 18.13

Créer une fonction `pre_traitement(motif)` qui prend en paramètre une chaîne de caractères `motif` et qui renvoie un dictionnaire contenant le décalage à effectuer.
Par exemple, pour le mot `ulyse`, la fonction va renvoyer le dictionnaire `{'u':0, 'l':1, 'y':2, 's':4}`. Comme sur l'exemple, on ne considérera pas la dernière lettre du mot. Lorsqu'il y a plusieurs lettres identiques, on s'intéressera seulement à celle la plus à droite (sauf la dernière lettre).

18.3.5 Comment utiliser le pré traitement pour l'algorithme de Boyer-Moore en prenant en compte la règle du mauvais caractère ?

On reprend l'exemple précédent

Le pré-traitement du motif donnerait `{'v':1, 't':3, 'a':2}`

texte :	p	x	y	o	t	a	v	a	t	a	v	a	t	i_texte	i_motif	comparaison	temps
motif :	v	a	t	a	v	a								5	5	'a' == 'a'	1
	v	a	t	a	v	a								4	4	't' != 'v'	2
			v	a	t	a	v	a						7	5	'a' == 'a'	3
			v	a	t	a	v	a						6	4	'v' == 'v'	4
			v	a	t	a	v	a						5	3	'a' == 'a'	5
			v	a	t	a	v	a						4	2	't' == 't'	6
			v	a	t	a	v	a						3	1	'o' != 'a'	7
				v	a	t	a	v	a					8	5	't' != 'a'	8
					v	a	t	a	v	a				11	5	'a' == 'a'	9
					v	a	t	a	v	a				10	4	'v' == 'v'	10
					v	a	t	a	v	a				9	3	'a' == 'a'	11
					v	a	t	a	v	a				8	2	't' == 't'	12
					v	a	t	a	v	a				7	1	'a' == 'a'	13
					v	a	t	a	v	a				6	0	'v' == 'v'	14

- A la deuxième comparaison, on a un mauvais caractère 't'. 't' se trouve dans le dictionnaire et sa valeur est 2. L'indice du 'mauvais caractère' dans le motif est 4. On va donc décaler de 4 - 2, soit de 2 crans.
- A la 7ème comparaison, 'o' est un mauvais caractère. 'o' ne se trouve pas dans le dictionnaire, et on va affecter le nombre -1. L'indice du mauvais caractère dans le motif est 1. On décale donc de 1 - -1, soit de 2 crans.
- A la 8ème comparaison, 't' est un mauvais caractère. 't' se trouve dans le dictionnaire et sa valeur est 2. L'indice du 'mauvais caractère' dans le motif est 5. On décale donc de 5 - 2, soit de 3 crans.

18.3.6 L'algorithme de Boyer Moore avec la règle du mauvais caractère uniquement

L'algorithme

```

1  Fonction CreerTableMauvaisCaractere(motif)
2      table  $\leftarrow$  dictionnaire vide
3      m  $\leftarrow$  longueur(motif)
4      for i  $\leftarrow$  0 to m - 2 do
5          | table[motif[i]]  $\leftarrow$  i
6      end
7      return table
8  fin

9  Fonction RechercheBoyerMoore(texte, motif)
10     n  $\leftarrow$  longueur(texte)
11     m  $\leftarrow$  longueur(motif)
12     occurrences  $\leftarrow$  liste vide
13     if m = 0 ou n < m then return occurrences
14     table  $\leftarrow$  CreerTableMauvaisCaractere(motif)
15     i  $\leftarrow$  0
16     while i  $\leq$  n - m do
17         | j  $\leftarrow$  m - 1
18         | while j  $\geq$  0 et motif[j] = texte[i + j] do
19             | j  $\leftarrow$  j - 1
20         | end
21         | if j < 0 then
22             | Ajouter i à occurrences
23             | if i + m < n then
24                 | cSuivant  $\leftarrow$  texte[i + m]
25                 | if cSuivant  $\in$  table then
26                     | derniereOcc  $\leftarrow$  table[cSuivant]
27                 | else
28                     | derniereOcc  $\leftarrow$  -1
29                 | end
30                 | i  $\leftarrow$  i + max(1, m - derniereOcc)
31             | else
32                 | i  $\leftarrow$  i + 1
33             | end
34         | else
35             | cFautif  $\leftarrow$  texte[i + j]
36             | if cFautif  $\in$  table then
37                 | derniereOcc  $\leftarrow$  table[cFautif]
38             | else
39                 | derniereOcc  $\leftarrow$  -1
40             | end
41             | i  $\leftarrow$  i + max(1, j - derniereOcc)
42         | end
43     end
44     return occurrences
45 fin

```

Algorithme 4: Algorithme de Boyer-Moore simplifié (Règle du mauvais caractère)

Implémentation Python

● Exercice 18.14

- | Implémenter cet algorithme en Python. Vérifier avec les exemples précédents.

Remarque

Nous n'avons présenté qu'une version simplifiée de l'algorithme complet. L'algorithme complet de Boyer-Moore utilise une deuxième table de décalage, beaucoup plus difficile à calculer, qui permet de tenir compte des caractéristiques du motif dans le cas où celui-ci présente des similarités internes, ce qui permet d'effectuer des décalages plus importants, donc d'augmenter l'efficacité de la recherche. L'algorithme complet de Boyer-Moore présente des difficultés en termes de justification et de programmation effective qui dépassent le niveau attendu en NSI. C'est pourquoi nous ne l'évoquons pas ici. Le lecteur curieux pourra lire les pages 360–366 de l'ouvrage de Berstel, Beauquier et Chrétienne, disponible en ligne : [ici](#)